

Security Incident Report

Obfuscated malware loader discovered in `babel.config.js` — neotree-react-native-app

SEVERITY: CRITICAL

Repository	neotree-react-native-app
Affected file	<code>babel.config.js</code> (repo root)
Branch investigated	<code>optimise-qr-code</code> (payload confirmed as an ancestor of <code>HEAD</code> ; very likely present on <code>master</code> / <code>stage</code> as well since the introducing commit long predates this branch)
Report date	2026-08-15
Prepared by	Claude Code, at the request of Morris (morris@neotree.org)

1. Summary

While working on an unrelated QR-code scanning improvement, an inspection of `babel.config.js` (a routine step while checking the project's build configuration) revealed a large block of obfuscated JavaScript appended after the file's normal, legitimate `module.exports`. Once safely decoded (see Section 3), this code turned out to be a **multi-stage malware downloader / backdoor** that fetches and executes attacker-controlled code on any machine that runs Babel for this project — i.e. any developer running `expo start`, `expo run:android`, `expo run:ios`, tests, or any CI/EAS build.

The malicious code is already committed to git history and has been carried forward, unchanged, through every subsequent commit that touched this file, up to and including the current `HEAD` of this branch.

2. How this was discovered

1. While reviewing the project's Android build configuration, `babel.config.js` was opened to check the existing Babel plugin setup.
2. The file's tail contained an extremely long single line: legitimate code (`module.exports = function (api) { ... }`), followed by hundreds of blank/tab characters, followed by a dense block of obfuscated JavaScript — a pattern with no legitimate reason to exist in a Babel config file.
3. The obfuscation used a recognisable "self-decoding string cipher" shape: helper functions that shuffle and substitute characters in a ciphertext string to reconstruct plaintext at runtime, then dynamically compile that plaintext into an executable function.
4. Rather than running the code (which would have executed whatever the code does), a decode-only harness was built in an isolated scratch directory. This harness copied only the file's own pure string-decoding helper functions (verified by inspection to contain no `require`, `filesystem`, `network`, or `eval` calls themselves) and ran *only* those steps, deliberately stopping the process one line before the point where the decoded text would normally be compiled into a function and invoked. This produced the fully decoded payload as plain text, without ever letting it execute.
5. `git log`, `git blame`, and `git grep` across all branches were then used to establish when the code was introduced, by whom, and whether it appears anywhere else in the repository.

Provenance findings:

- Introduced in commit `fc1e715` — *"show admission weight if birthweight is not found"*, authored by **KumaloWilson <kumalowilson900@gmail.com>**, dated 2026-04-22. This commit also touched an unrelated file (`src/Home/Script/Start/field/search.tsx`) — consistent with the malicious change being smuggled inside a legitimate-looking feature commit, a common sign of a compromised developer machine or tooling rather than a deliberate, hand-crafted insider commit.
- This commit is an ancestor of the current `HEAD` , so the payload is present in the working tree today.
- A later commit, `b5963dd` ("export legacy optimizations", same author, 2026-08-12), does strip this exact payload back out — but only on a separate, unmerged remote branch (`origin/NEOAPP1412`). That fix never reached `master` / `stage` /this branch.
- A repository-wide search across every commit on every branch found the malicious marker **only** in `babel.config.js` — no other file in the project's history is affected.

3. What the malware actually does

The obfuscation is layered (a character-shuffle cipher, a dictionary/table-substitution decoder, and a trick of reading `someFunction.constructor` to obtain JavaScript's built-in `Function` constructor without ever writing the word "Function" — specifically to evade naive text-based malware scanners). After safely peeling back every layer, the final, real behaviour is as follows:

3.1 Self-throttling

On each run, it stores a timestamp on Node's global object and exits immediately if it last ran less than 30 seconds ago — reducing its visibility and avoiding redundant network activity if `babel.config.js` is loaded repeatedly (e.g. by a file watcher).

3.2 "Dead-drop" command-and-control resolution

Instead of a hardcoded C2 domain or IP address (which could be found and blocked), the code:

- Makes an HTTP request to a blockchain explorer-style API, and a JSON-RPC request to a blockchain node endpoint, to read the data field of a specific transaction on a public blockchain;
- XOR-decodes that transaction data using the resolved wallet address itself as the decryption key.

This is a known technique sometimes called a **blockchain dead-drop resolver**: it lets the attacker publish a new C2 location at any time simply by broadcasting a new transaction, and lets every already-deployed copy of the malware find the current location dynamically — without ever changing the malware's own source code, and without giving defenders a fixed domain/IP to block.

3.3 Remote code execution

Using the address/location resolved in 3.2, the code fetches a remote text payload over HTTP and runs it directly with `eval()` — full arbitrary JavaScript execution, with whatever OS-level privileges the current process has.

3.4 Persistence via a detached background process

Separately, it fetches a second payload and launches it in a new child process configured with:

```
detached: true, stdio: 'ignore', windowsHide: true
```

This spawns a process that keeps running independently of the parent (so closing the terminal/IDE does not stop it), discards its input/output (so nothing shows in any visible console), and hides its window on Windows — a standard backdoor/persistence pattern.

Important nuance: this code is a *loader*, not the final malicious payload. It does not itself steal data or mine cryptocurrency — it fetches and blindly executes whatever the attacker's infrastructure serves at the exact moment it runs. That means its real-world impact was decided by the attacker at runtime and may have changed over time; it cannot be fully characterised from the static code alone.

4. Impact to this codebase and its developers

- **Trigger:** this code runs as a side effect of Node.js loading `babel.config.js` — i.e. any time Metro/Babel processes this project: `expo start`, `expo run:android`, `expo run:ios`, `yarn test`, or any CI/EAS Build run.
- **Who is affected:** any developer machine, or CI/EAS build runner, that checked out and *ran* (not merely viewed) this repository at commit `fc1e715` or any descendant of it, on any branch carrying that commit, at any point since 2026-04-22.
- **What is at risk:** because the payload executes with the full privileges of whichever user/process ran it, any credential, token, SSH key, cloud credential, or file accessible in that session should be treated as potentially exposed — not only material related to this repository.
- **Persistence:** because of the detached background process (3.4), simply closing the app/terminal does not guarantee the malicious process has stopped; it may continue running, and re-fetching/executing new code, until the machine is rebooted or the process is explicitly found and killed.
- **Good news — end users are not directly affected:** `babel.config.js` is only ever read by Babel during the JavaScript build/transform step. It is never bundled into the compiled app that ships to phones/tablets. This means the malware runs only in developer and CI/build environments, not on any Neotree device running the finished app.

5. Steps to keep this codebase safe

☐	Strip the obfuscated block from <code>babel.config.js</code> on every branch that carries it — confirmed so far on the history leading to this branch's <code>HEAD</code> ; <code>master</code> and <code>stage</code> should be checked and fixed the same way.
☐	Re-run the same repository-wide search (<code>git grep</code> across all commits/branches for the same obfuscation markers, and more generally for other minified single-line blocks, <code>eval(</code> , or <code>.constructor</code> -based dynamic <code>Function</code> construction) in case similar code was injected elsewhere and simply wasn't caught by this specific marker.
☐	Add a CI check that fails the build if config files (<code>babel.config.js</code> , <code>metro.config.js</code> , <code>jest.config.js</code> , etc.) contain suspicious patterns — e.g. abnormally long single lines, <code>eval(</code> , or dynamic <code>Function</code> construction.

☐	Treat all secrets reachable from any affected machine or CI runner as compromised: rotate GitHub credentials/SSH keys for the affected account, npm/yarn auth tokens, Expo/EAS access tokens, and any cloud (AWS/GCP/Azure) or database credentials used in local development or CI since 2026-04-22.
☐	Review GitHub organization audit logs for the affected account since 2026-04-22 for unusual activity (new SSH keys, new personal access tokens, unexpected pushes or collaborator/permission changes).
☐	Review CI/EAS Build history since 2026-04-22 for anomalies (unusual network activity, unexpected build durations).
☐	Decide, as an organisation, whether to rewrite git history to purge the payload entirely (disruptive: requires a coordinated force-push and every collaborator re-cloning) versus fixing it forward on the default branch only, understanding the payload remains recoverable from history in the latter case.
☐	Ask all collaborators to pull the fixed branch once available, and to check their own machines for the signs listed in Section 6.

6. What the machine owner (and anyone else who ran the affected code) should do

☐	Assume compromise: disconnect the affected machine from sensitive networks/VPNs until it has been checked.
☐	Look for and terminate unexpected background processes — especially detached <code>node</code> processes with no obvious parent terminal window. On macOS/Linux: <code>ps aux grep node</code> . On Windows: Task Manager → Details tab, look for orphaned <code>node.exe</code> processes.
☐	Run a full antivirus/anti-malware scan (Windows Defender full scan, Malwarebytes, or the macOS/Linux equivalent).
☐	Rotate every credential accessible from that machine's user session: git/GitHub SSH keys and personal access tokens, npm/yarn tokens, cloud CLI credentials, and Expo/EAS tokens.
☐	Check for unauthorized entries in <code>~/.ssh/authorized_keys</code> , and for unexpected scheduled tasks / cron jobs / systemd services (Linux) / LaunchAgents (macOS) / Scheduled Tasks (Windows) — common secondary persistence locations besides the detached process this malware sets up itself.
☐	Check shell startup files (<code>.bashrc</code> , <code>.zshrc</code> , <code>.bash_profile</code>) and editor/IDE startup configuration for unexpected additions.
☐	Change the password on the <code>kumalowilson900@gmail.com</code> -linked npm/GitHub accounts and enable two-factor authentication if not already active; review the account for any packages published or sessions created without the owner's knowledge.
☐	Given this is a full arbitrary-code-execution loader whose actual second-stage behaviour is attacker-controlled and unknown, a clean OS reinstall should be considered if the organisation's risk tolerance is low.

This report was produced by decoding the malicious payload in a fully isolated, offline scratch environment. At no point was the decoded payload executed, compiled into a function, or allowed network/filesystem access beyond reading the single source file being analysed.